

Od chaosu plików do czystej ramki danych

Obróbka danych — `pathlib`, `glob`, `re`, `pandas`.str

Cel zajęć: mamy katalog z pomiarami środowiskowymi z kilku stacji w Łodzi. Pliki są w różnych podkatalogach, mają różne formaty nazw, w środku różne separatory. Naszym zadaniem jest **wczytać wszystko, wyciągnąć metadane z nazw plików** (stacja, data, typ pomiaru) i **połączyć w jedną ramkę** gotową do analizy.

```
In [41]: import zipfile
from pathlib import Path

zip_path = Path("dane_lodz.zip")
output_dir = Path("dane_lodz")

with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall(output_dir)
```

1. Rozpoznanie terenu

Zacznijmy od obejrzenia, z czym mamy do czynienia.

```
In [42]: from pathlib import Path

ROOT = Path("dane_lodz/dane_lodz")

# Sprawdzenie, czy katalog istnieje
if ROOT.exists() and ROOT.is_dir():
    # Lista podkatalogów
    for p in ROOT.iterdir():
        print(p)
else:
    print(f"Katalog '{ROOT}' nie istnieje.")
```

```
dane_lodz/dane_lodz/wilgotnosc
dane_lodz/dane_lodz/jakosc_powietrza
dane_lodz/dane_lodz/temperatura
```

```
In [43]: # Pokaż drzewo - kilka pierwszych plików z każdego katalogu
for subdir in sorted(ROOT.iterdir()):
    if subdir.is_dir():
        print(f"\n📁 {subdir.name}/")
        for f in sorted(subdir.iterdir())[:3]:
            print(f"    {f.name}")
        print("    ...")
```

```

jakosc_powietrza/
info.txt
pm25-ST01_polesie-20250301.csv
pm25-ST01_polesie-20250302.csv
...

temperatura/
README.md
_old_backup.csv.bak
temp_ST01_polesie_2025-03-01.csv
...

wilgotnosc/
humidity_ST01_polesie_v1_2025_03_01.txt
humidity_ST01_polesie_v1_2025_03_02.txt
humidity_ST01_polesie_v1_2025_03_04.txt
...

```

2. pathlib — podstawy

`pathlib` zastępuje `os.path`. Główna klasa: `Path`. Działa **cross-platform** — używaj `/` zamiast `os.path.join`.

```

In [44]: # Operator / buduje ścieżki
plik = ROOT / "temperatura" / "temp_ST01_polesie_2025-03-01.csv"
print("Pełna ścieżka: ", plik)
print("Nazwa pliku: ", plik.name)
print("Sama nazwa: ", plik.stem)
print("Rozszerzenie: ", plik.suffix)
print("Katalog nadrzędny:", plik.parent)
print("Czy istnieje: ", plik.exists())

```

```

Pełna ścieżka: dane_lodz/dane_lodz/temperatura/temp_ST01_polesie_2025-03-01.csv
Nazwa pliku: temp_ST01_polesie_2025-03-01.csv
Sama nazwa: temp_ST01_polesie_2025-03-01
Rozszerzenie: .csv
Katalog nadrzędny: dane_lodz/dane_lodz/temperatura
Czy istnieje: True

```

Ćwiczenie 1

Dla pliku `plik` powyżej wypisz:

- nazwę bez rozszerzenia
- nazwę katalogu nadrzędnego (samą nazwę, nie pełną ścieżkę)
- rozmiar pliku w bajtach (podpowiedź: `.stat().st_size`)

```

In [59]: import pandas as pd
import re
from pathlib import Path

ROOT = Path("dane_lodz")

def wczytaj_temperature(root):
    wzorzec = re.compile(r"temp_(?P<stacja>ST\d{2})_([a-z])+(?P<data>\d{4}-\d{2}-

```

```

ramki = []

for plik in Path(root).rglob("temperatura/*.csv"):
    m = wzorzec.match(plik.name)
    if m:
        df = pd.read_csv(plik)
        df['stacja'] = m.group('stacja')
        ramki.append(df)

if ramki:
    return pd.concat(ramki, ignore_index=True)
return pd.DataFrame()

temp_df = wczytaj_temperature(ROOT)
print("Kształt ramki temperatury:", temp_df.shape)
temp_df.head()

```

Kształt ramki temperatury: (272, 4)

Out[59]:

	timestamp	temperatura_C	uwagi	stacja
0	2025-03-05 00:00:00	2.2	NaN	ST02
1	2025-03-05 03:00:00	5.97	ok	ST02
2	2025-03-05 06:00:00	19.28	NaN	ST02
3	2025-03-05 09:00:00	17.99	NaN	ST02
4	2025-03-05 12:00:00	19.23 C	deszcz	ST02

3. glob i rglob — znajdowanie plików

- `Path.glob(pattern)` — w danym katalogu (jeden poziom)
- `Path.rglob(pattern)` — **rekurencyjnie** (we wszystkich podkatalogach)
- wzorce: `*` (cokolwiek), `?` (jeden znak), `[abc]` (zbiór znaków)

```

In [45]: # Wszystkie pliki CSV w całym drzewie
csv_files = list(ROOT.rglob("*.csv"))
print(f"Znaleziono plików CSV: {len(csv_files)}")
for f in csv_files[:5]:
    print(" ", f)

```

Znaleziono plików CSV: 80

```

dane_lodz/dane_lodz/jakosc_powietrza/pm25-ST04_gorna-20250301.csv
dane_lodz/dane_lodz/jakosc_powietrza/pm25-ST03_baluty-20250305.csv
dane_lodz/dane_lodz/jakosc_powietrza/pm25-ST04_gorna-20250308.csv
dane_lodz/dane_lodz/jakosc_powietrza/pm25-ST02_widzew-20250302.csv
dane_lodz/dane_lodz/jakosc_powietrza/pm25-ST01_polesie-20250310.csv

```

```

In [46]: # Tylko pliki z temperatury
temp_files = list((ROOT / "temperatura").glob("temp_*.csv"))
print(f"Plików temperatury: {len(temp_files)}")

```

Plików temperatury: 40

```

In [47]: # UWAGA: glob łapie też śmieci - np. backupy. Trzeba filtrować.
podejrzane = list(ROOT.rglob("*backup*"))
print("Pliki które chcemy zignorować:")

```

```
for f in podejrzone:
    print(" ", f)
```

Pliki które chcemy zignorować:

dane_lodz/dane_lodz/temperatura/_old_backup.csv.bak

Ćwiczenie 2

Znajdź:

1. Wszystkie pliki `.txt` w drzewie
2. Wszystkie pliki PM2.5 (zaczynają się od `pm25-`)
3. Wszystkie pliki temperatury, **z wyjątkiem** wersji `_v2`

```
In [67]: from pathlib import Path

ROOT = Path("dane_lodz")

pliki_txt = list(ROOT.rglob("*.txt"))
print("Znalezione pliki .txt:", len(pliki_txt))

pliki_pm25 = list(ROOT.rglob("pm25-*"))
print("Znalezione pliki PM2.5:", len(pliki_pm25))

pliki_temp_bez_v2 = [plik for plik in ROOT.rglob("temp_*") if "_v2" not in plik]
print("Znalezione pliki temperatury (bez v2):", len(pliki_temp_bez_v2))
```

Znalezione pliki .txt: 41

Znalezione pliki PM2.5: 40

Znalezione pliki temperatury (bez v2): 34

4. Wyrażenia regularne — krótkie powtórzenie

Regex to **język opisywania wzorców tekstu**. Najważniejsze elementy:

Wzorzec	Znaczenie
<code>\d</code>	cyfra (0-9)
<code>\w</code>	znak "słowny" (litera, cyfra, <code>_</code>)
<code>\s</code>	biały znak
<code>.</code>	dowolny znak
<code>+</code>	jeden lub więcej
<code>*</code>	zero lub więcej
<code>{n}</code>	dokładnie n
<code>{n,m}</code>	od n do m
<code>[abc]</code>	jeden ze znaków
<code>(...)</code>	grupa
<code>(?P<nazwa>...)</code>	grupa nazwana ← KLUCZOWE

Funkcje z `re`:

- `re.search(pattern, text)` — szuka **gdziekolwiek** w tekście
- `re.match(pattern, text)` — szuka **od początku**
- `re.findall(pattern, text)` — wszystkie dopasowania
- `re.sub(pattern, replacement, text)` — zamiana

In [48]: `import re`

```
# Prosty przykład: wyciągnij datę z nazwy pliku
nazwa = "temp_ST01_polesie_2025-03-01.csv"
wzorzec = r"(\d{4}-\d{2}-\d{2})"
m = re.search(wzorzec, nazwa)
print(m.group(1))
```

2025-03-01

Grupy nazwane to klucz do czytelnego kodu. Zamiast `m.group(1)`, `m.group(2)` ... mamy `m.group('data')`.

```
In [49]: # Wzorzec dla plików temperatury:
# temp_<STACJA>_<DATA>.csv lub temp_<STACJA>_<DATA>_v2.csv
wzorzec_temp = re.compile(
    r"temp_(?P<stacja>ST\d{2}_\w+?)(?P<data>\d{4}-\d{2}-\d{2})(?:_v(?P<wersja>\d+))?"
)

for f in list((ROOT / "temperatura").glob("temp_*.csv"))[:5]:
    m = wzorzec_temp.match(f.name)
    if m:
        print(f.name)
        print(" →", m.groupdict())
```

```
temp_ST02_widzew_2025-03-05.csv
→ {'stacja': 'ST02_widzew', 'data': '2025-03-05', 'wersja': None}
temp_ST04_gorna_2025-03-07.csv
→ {'stacja': 'ST04_gorna', 'data': '2025-03-07', 'wersja': None}
temp_ST03_baluty_2025-03-06.csv
→ {'stacja': 'ST03_baluty', 'data': '2025-03-06', 'wersja': None}
temp_ST02_widzew_2025-03-08.csv
→ {'stacja': 'ST02_widzew', 'data': '2025-03-08', 'wersja': None}
temp_ST03_baluty_2025-03-08.csv
→ {'stacja': 'ST03_baluty', 'data': '2025-03-08', 'wersja': None}
```

Rozbiór wzorca:

- `temp_` — literalny początek
- `(?P<stacja>ST\d{2}_\w+?)` — `ST` + 2 cyfry + `_` + nazwa (leniwa, żeby nie zjeść daty)
- `_` — separator
- `(?P<data>\d{4}-\d{2}-\d{2})` — data w formacie ISO
- `(?:_v(?P<wersja>\d+))?` — **opcjonalna** wersja, `(?:...)` to grupa nieprzechwytyjąca
- `\.csv` — kropka literalna (uciekamy `\.`) + rozszerzenie

ćwiczenie 3

Napisz regex dla plików PM2.5 (przykład: `pm25-ST01_polesie-20250301.csv`).

Wyciągnij stację i datę. Uwaga: data nie ma separatorów!

I drugi — dla wilgotności (np. `humidity_ST02_widzew_v1_2025_03_05.txt`). Tu wersja jest **w środku**, nie na końcu.

```
In [61]: import re

# 1. Regex dla plików PM2.5 (data bez separatorów)
wzorzec_pm25 = re.compile(r"pm25-(?P<stacja>ST\d{2}_\w+)-(?P<data>\d{8})\.csv")

print("--- Test PM2.5 ---")
test_pm = "pm25-ST01_polesie-20250301.csv"
m_pm = wzorzec_pm25.match(test_pm)
if m_pm:
    print(test_pm)
    print(" →", m_pm.groupdict())

# 2. Regex dla wilgotności (wersja w środku nazwy)
wzorzec_wilgotnosc = re.compile(r"humidity_(?P<stacja>ST\d{2}_\w+)_v(?P<wersja>\d+)_\d{8}\.txt")

print("\n--- Test Wilgotność ---")
test_wilg = "humidity_ST02_widzew_v1_2025_03_05.txt"
m_wilg = wzorzec_wilgotnosc.match(test_wilg)
if m_wilg:
    print(test_wilg)
    print(" →", m_wilg.groupdict())

--- Test PM2.5 ---
pm25-ST01_polesie-20250301.csv
  → {'stacja': 'ST01_polesie', 'data': '20250301'}

--- Test Wilgotność ---
humidity_ST02_widzew_v1_2025_03_05.txt
  → {'stacja': 'ST02_widzew', 'wersja': '1', 'data': '2025_03_05'}
```

5. Łączymy: pathlib + regex + pandas

Teraz właściwa robota. Strategia:

1. Znajdź pliki danego typu (`rglob` + regex do walidacji nazwy)
2. Wyciągnij metadane z nazwy
3. Wczytaj do pandas
4. Dodaj kolumny z metadanymi
5. Złóż wszystko w jedną ramkę

```
In [50]: import pandas as pd

def wczytaj_temperature(root: Path) -> pd.DataFrame:
    """Wczytuje wszystkie pliki temperatury, dodaje metadane, łączy w jedną ramkę"""
    ramki = []
    for f in root.rglob("temp_*.csv"):
        m = wzorzec_temp.match(f.name)
        if not m:
```

```

        print(f"⚠ Pomijam: {f.name}")
        continue
    meta = m.groupdict()
    df = pd.read_csv(f)
    df["stacja"] = meta["stacja"]
    df["data_pliku"] = meta["data"]
    df["wersja"] = meta.get("wersja") or "1"
    df["plik"] = f.name
    ramki.append(df)
    return pd.concat(ramki, ignore_index=True)

temp_df = wczytaj_temperature(ROOT)
print("Kształt:", temp_df.shape)
temp_df.head()

```

Kształt: (320, 7)

Out[50]:

	timestamp	temperatura_C	uwagi	stacja	data_pliku	wersja	
0	2025-03-05 00:00:00	2.2	NaN	ST02_widzew	2025-03-05	1	temp_ST02_widzew C
1	2025-03-05 03:00:00	5.97	ok	ST02_widzew	2025-03-05	1	temp_ST02_widzew C
2	2025-03-05 06:00:00	19.28	NaN	ST02_widzew	2025-03-05	1	temp_ST02_widzew C
3	2025-03-05 09:00:00	17.99	NaN	ST02_widzew	2025-03-05	1	temp_ST02_widzew C
4	2025-03-05 12:00:00	19.23 C	deszcz	ST02_widzew	2025-03-05	1	temp_ST02_widzew C

In [51]:

```

print("Typ kolumny:", temp_df["temperatura_C"].dtype)
print("\nPodejrzane wartości:")
podejrzane = temp_df[temp_df["temperatura_C"].astype(str).str.contains("C", na=F
podejrzane.head()

```

Typ kolumny: object

Podejrzane wartości:

Out[51]:

	timestamp	temperatura_C	uwagi	stacja	data_pliku	wersja	
4	2025-03-05 12:00:00	19.23 C	deszcz	ST02_widzew	2025-03-05	1	temp_ST02_wic
40	2025-03-01 00:00:00	4.27 C	ok	ST02_widzew	2025-03-01	2	temp_ST02_wic 0:
43	2025-03-01 09:00:00	2.07 C	NaN	ST02_widzew	2025-03-01	2	temp_ST02_wic 0:
63	2025-03-06 21:00:00	23.24 C	ok	ST01_polesie	2025-03-06	1	temp_ST01_po
103	2025-03-07 21:00:00	23.26 C	deszcz	ST03_baluty	2025-03-07	1	temp_ST03_ba

6. `pandas.Series.str` — czyszczenie kolumn tekstowych

Akcesor `.str` w pandas to **odpowiednik metod stringa, ale zvektoryzowany**.

Najważniejsze:

```
In [52]: # Konwersja na string i usunięcie tekstu + spacji
temp_df["temperatura_C"] = (
    temp_df["temperatura_C"]
    .astype(str)
    .str.replace("C", "", regex=False) # usuń literę C
    .str.strip()                       # usuń białe znaki z brzegów
    .astype(float)                     # teraz można rzutować
)

print("Typ po czyszczeniu:", temp_df["temperatura_C"].dtype)
print(temp_df["temperatura_C"].describe())
```

Typ po czyszczeniu: float64

```
count    320.000000
mean      10.938094
std        6.454670
min       -1.750000
25%         5.570000
50%        10.695000
75%        16.310000
max        24.250000
```

Name: temperatura_C, dtype: float64

`str.extract` z grupami nazwanymi — najpotężniejsza metoda

Wyobraźcie sobie, że chcemy **rozbić nazwę stacji** `ST01_polesie` na kod numeryczny i nazwę dzielnicy:

```
In [53]: rozbita = temp_df["stacja"].str.extract(r"ST(?<kod>\d+)(?<dzielnica>\w+)")
rozbita.head()
```

Out[53]:

	kod	dzielnica
0	02	widzew
1	02	widzew
2	02	widzew
3	02	widzew
4	02	widzew

```
In [54]: # Dołączamy te kolumny do głównej ramki
temp_df = pd.concat([temp_df, rozbita], axis=1)
temp_df.head()
```


Out[54]:

	timestamp	temperatura_C	uwagi	stacja	data_pliku	wersja	
0	2025-03-05 00:00:00	2.20	NaN	ST02_widzew	2025-03-05	1	temp_ST02_widzew C
1	2025-03-05 03:00:00	5.97	ok	ST02_widzew	2025-03-05	1	temp_ST02_widzew C
2	2025-03-05 06:00:00	19.28	NaN	ST02_widzew	2025-03-05	1	temp_ST02_widzew C
3	2025-03-05 09:00:00	17.99	NaN	ST02_widzew	2025-03-05	1	temp_ST02_widzew C
4	2025-03-05 12:00:00	19.23	deszcz	ST02_widzew	2025-03-05	1	temp_ST02_widzew C

ćwiczenie 4

Napisz funkcję `wczytaj_pm25(root)` analogiczną do `wczytaj_temperature`.

Uwaga:

- separator w plikach to `;` (parametr `sep` w `read_csv`)
- data w nazwie jest w formacie `YYYYMMDD` — sparsuj ją do `pd.Timestamp`
- kolumna `czas` w środku ma format `DD.MM.YYYY HH:MM` — sparsuj do `datetime`

In [63]:

```
import pandas as pd
import re
from pathlib import Path

def wczytaj_temperature(root):
    # Regex uwzględnia teraz stację z nazwą dzielnicy oraz opcjonalną wersję np.
    wzorzec = re.compile(r"temp_(?P<stacja>ST\d{2}_[a-zA-Z]+)_(?P<data>\d{4}-\d{2}-\d{2})")
    ramki = []

    # Szukamy wszystkich plików zaczynających się od temp_ w formacie csv
    for plik in Path(root).rglob("temp_*.csv"):
        m = wzorzec.match(plik.name)
        if m:
            df = pd.read_csv(plik)

            # Przypisanie wyciągniętych danych z nazwy pliku do kolumn
            for klucz, wartosc in m.groupdict().items():
                df[klucz] = wartosc

            df["data_pliku"] = pd.to_datetime(df["data"])
            ramki.append(df)

    return pd.concat(ramki, ignore_index=True) if ramki else pd.DataFrame()

# Wywołanie testowe - zmień "." na ścieżkę do folderu z danymi, jeśli masz je w
temp_df = wczytaj_temperature(".")
print("Wczytano wierszy:", len(temp_df))
temp_df.head()
```

Wczytano wierszy: 320

Out[63]:

	timestamp	temperatura_C	uwagi	stacja	data	data_pliku
0	2025-03-05 00:00:00	2.2	NaN	ST02_widzew	2025-03-05	2025-03-05
1	2025-03-05 03:00:00	5.97	ok	ST02_widzew	2025-03-05	2025-03-05
2	2025-03-05 06:00:00	19.28	NaN	ST02_widzew	2025-03-05	2025-03-05
3	2025-03-05 09:00:00	17.99	NaN	ST02_widzew	2025-03-05	2025-03-05
4	2025-03-05 12:00:00	19.23 C	deszcz	ST02_widzew	2025-03-05	2025-03-05

7. `pd.concat` z `keys` — MultiIndex na łączeniu

Czasem chcemy zachować informację, z **którego pliku** pochodzi wiersz. Można dodać kolumnę (jak wyżej), albo użyć MultiIndex:

In [55]:

```
kawalki = {}
for f in list((ROOT / "temperatura").glob("temp_*.csv"))[:3]:
    kawalki[f.stem] = pd.read_csv(f)

duza = pd.concat(kawalki, names=["plik_zrodlowy", "wiersz"])
duza.head(7)
```

Out[55]:

		timestamp	temperatura_C	uwagi
plik_zrodlowy wiersz				
temp_ST02_widzew_2025-03-05	0	2025-03-05 00:00:00	2.2	NaN
	1	2025-03-05 03:00:00	5.97	ok
	2	2025-03-05 06:00:00	19.28	NaN
	3	2025-03-05 09:00:00	17.99	NaN
	4	2025-03-05 12:00:00	19.23 C	deszcz
	5	2025-03-05 15:00:00	20.99	ok
	6	2025-03-05 18:00:00	19.44	NaN

8. Podsumowanie — pełny pipeline

Złożenie wszystkiego w jedną funkcję, która zwraca **gotową do analizy ramkę** ze wszystkich źródeł:

```

In [56]: def zbuduj_dataset(root: Path) -> pd.DataFrame:
        """Łączy temperatury i PM2.5 w jedną ramkę po (stacja, czas)."""
        temp = wczytaj_temperature(root)
        temp["timestamp"] = pd.to_datetime(temp["timestamp"])
        # Upewnij się, że kolumna 'temperatura_C' jest numeryczna
        temp["temperatura_C"] = (
            temp["temperatura_C"]
            .astype(str)
            .str.replace("C", "", regex=False)
            .str.strip()
            .astype(float)
        )
        # użyjemy zaokrąglenia do godziny - bo PM ma co 6h, temp co 3h
        temp["czas_h"] = temp["timestamp"].dt.floor("6h")

        pm = wczytaj_pm25(root) # funkcja z poprzedniego ćwiczenia
        pm["czas_h"] = pm["czas"].dt.floor("6h")

        # Agregacja temperatury do okien 6h
        temp_agg = temp.groupby(["stacja", "czas_h"], as_index=False).agg(
            temp_srednia=("temperatura_C", "mean")
        )

        polaczone = pm.merge(temp_agg, on=["stacja", "czas_h"], how="left")
        return polaczone[["stacja", "czas_h", "PM2.5_ug_m3", "PM10_ug_m3", "temp_sre

# Wzorzec dla plików PM2.5:
# pm25-<STACJA>-<DATA>.csv
wzorzec_pm = re.compile(r"pm25-(?P<stacja>ST\d{2}_\w+)-(?P<data>\d{8})\.csv")

def wczytaj_pm25(root: Path) -> pd.DataFrame:
    """Wczytuje wszystkie pliki PM2.5, dodaje metadane, łączy w jedną ramkę."""
    ramki = []
    for f in root.rglob("pm25-*.csv"):
        m = wzorzec_pm.match(f.name)
        if not m:
            print(f"⚠ Pomijam: {f.name}")
            continue
        meta = m.groupdict()
        df = pd.read_csv(f, sep=';') # Użyj separatora średnika
        df["stacja"] = meta["stacja"]
        df["data_pliku"] = pd.to_datetime(meta["data"], format="%Y%m%d") # Parsu
        df["plik"] = f.name
        df["czas"] = pd.to_datetime(df["czas"], format="%d.%m.%Y %H:%M") # Parsu
        ramki.append(df)

    if not ramki:
        print(f"Brak plików 'pm25-*.csv' w katalogu '{root}' lub jego podkatalog
        return pd.DataFrame() # Return an empty DataFrame if no files were found

    return pd.concat(ramki, ignore_index=True)

final = zbuduj_dataset(ROOT)
print("Kształt finalnej ramki:", final.shape)
final.head(10)

```

Kształt finalnej ramki: (160, 5)

Out[56]:

	stacja	czas_h	PM2.5_ug_m3	PM10_ug_m3	temp_srednia
0	ST04_gorna	2025-03-01 00:00:00	30.5	36.7	6.650
1	ST04_gorna	2025-03-01 06:00:00	70.2	116.0	12.905
2	ST04_gorna	2025-03-01 12:00:00	35.1	46.1	9.875
3	ST04_gorna	2025-03-01 18:00:00	52.5	64.3	11.355
4	ST03_baluty	2025-03-05 00:00:00	36.2	46.2	7.315
5	ST03_baluty	2025-03-05 06:00:00	37.0	64.1	19.185
6	ST03_baluty	2025-03-05 12:00:00	33.1	43.8	11.935
7	ST03_baluty	2025-03-05 18:00:00	74.2	93.0	10.875
8	ST04_gorna	2025-03-08 00:00:00	48.2	80.0	2.140
9	ST04_gorna	2025-03-08 06:00:00	58.2	89.3	8.470

Zadanie

Dopisz funkcję `wczytaj_wilgotnosc(root)` dla plików w `wilgotnosc/`. Uwagi:

1. Pliki mają **nagłówek z komentarzami** (linie zaczynające się od `#`) — pominięte w `read_csv` parametrem `comment="#"`.
2. Data w nazwie jest w formacie `YYYY_MM_DD` (podkreślniki!) — przekonwertuj.
3. Niektóre pliki mają wersję `v2` — w finalnej ramce zostaw tylko **najnowszą wersję** dla każdej (stacja, data).
4. Dołącz wilgotność do `final` z poprzedniego kroku.

Bonus: wykryj duplikaty pliku temperatury (`_v2`) i zostaw tylko nowszą wersję — analogicznie.

In [66]:

```
import pandas as pd
import re
from pathlib import Path

# Wzorzec dla plików PM2.5:
# pm25-<STACJA>-<DATA>.csv
wzorzec_pm = re.compile(r"pm25-(?P<stacja>ST\d{2}_\w+)-(?P<data>\d{8})\.csv")

def wczytaj_pm25(root: Path) -> pd.DataFrame:
    """Wczytuje wszystkie pliki PM2.5, dodaje metadane, łączy w jedną ramkę."""
    ramki = []
    for f in root.rglob("pm25-*.csv"):
        m = wzorzec_pm.match(f.name)
        if not m:
            print(f"⚠ Pomijam: {f.name}")
            continue
        meta = m.groupdict()
        df = pd.read_csv(f, sep=';') # Użyj separatora średnika
        df["stacja"] = meta["stacja"]
        df["data_pliku"] = pd.to_datetime(meta["data"], format="%Y%m%d") # Parsu
```

```

df["plik"] = f.name
df["czas"] = pd.to_datetime(df["czas"], format="%d.%m.%Y %H:%M") # Parsu
ramki.append(df)

if not ramki:
    print(f"Brak plików 'pm25-*.csv' w katalogu '{root}' lub jego podkatalog
    return pd.DataFrame() # Return an empty DataFrame if no files were found

return pd.concat(ramki, ignore_index=True)

def wczytaj_wilgotnosc(root):
    wzorzec = re.compile(r"humidity_(?P<stacja>ST\d{2}_[a-zA-Z]+)_v(?P<wersja>\d{4})")
    ramki = []

    for plik in Path(root).rglob("humidity_*.txt"):
        m = wzorzec.match(plik.name)
        if m:
            df = pd.read_csv(plik, comment='#')

            for klucz, wartosc in m.groupdict().items():
                df[klucz] = wartosc

            # Konwersja kolumny 'humidity_pct' do float
            df['humidity_pct'] = df['humidity_pct'].astype(float)

            df['data'] = pd.to_datetime(df['data'], format='%Y_%m_%d')
            df['wersja'] = df['wersja'].astype(int)
            df['czas'] = df['data'] + pd.to_timedelta(df['hour'], unit='h')

            ramki.append(df)

    if ramki:
        df_all = pd.concat(ramki, ignore_index=True)
        df_all = df_all.sort_values('wersja').drop_duplicates(subset=['stacja'],
        return df_all

    return pd.DataFrame()

def wczytaj_temperature(root):
    wzorzec = re.compile(r"temp_(?P<stacja>ST\d{2}_[a-zA-Z]+)_(?P<data>\d{4})-\d{4}")
    ramki = []

    for plik in Path(root).rglob("temp_*.csv"):
        m = wzorzec.match(plik.name)
        if m:
            df = pd.read_csv(plik)

            for klucz, wartosc in m.groupdict().items():
                # Ensure 'wersja' is handled robustly, defaulting to 1 if not pr
                if klucz == 'wersja':
                    df[klucz] = int(wartosc) if wartosc is not None else 1
                else:
                    df[klucz] = wartosc

            # If 'wersja' was not in groupdict at all, it won't be in df. Handle
            if 'wersja' not in df.columns:
                df['wersja'] = 1

            df['data_pliku'] = pd.to_datetime(df['data'])

```

```

        ramki.append(df)

    if ramki:
        df_all = pd.concat(ramki, ignore_index=True)
        df_all = df_all.sort_values('wersja').drop_duplicates(subset=['stacja'],
        return df_all

    return pd.DataFrame()

def zbuduj_dataset(root):
    temp = wczytaj_temperature(root)
    if not temp.empty:
        # Ensure 'temperatura_C' is numeric before aggregation
        temp["temperatura_C"] = (
            temp["temperatura_C"]
            .astype(str)
            .str.replace("C", "", regex=False)
            .str.strip()
            .astype(float)
        )
        temp["czas_h"] = pd.to_datetime(temp["timestamp"]).dt.floor("6h")
        temp_agg = temp.groupby(["stacja", "czas_h"], as_index=False).agg(
            temp_srednia=("temperatura_C", "mean")
        )
    else:
        temp_agg = pd.DataFrame(columns=["stacja", "czas_h", "temp_srednia"])

    pm = wczytaj_pm25(root)
    if not pm.empty:
        pm["czas_h"] = pd.to_datetime(pm["czas"]).dt.floor("6h")
    else:
        pm = pd.DataFrame(columns=["stacja", "czas_h"])

    wilg = wczytaj_wilgotnosc(root)
    if not wilg.empty:
        wilg["czas_h"] = wilg["czas"].dt.floor("6h")
        wilg_agg = wilg.groupby(["stacja", "czas_h"], as_index=False).agg(
            wilgotnosc_srednia=("humidity_pct", "mean")
        )
    else:
        wilg_agg = pd.DataFrame(columns=["stacja", "czas_h", "wilgotnosc_srednia"])

    polaczone = pm.merge(temp_agg, on=["stacja", "czas_h"], how="left")
    polaczone = polaczone.merge(wilg_agg, on=["stacja", "czas_h"], how="left")

    kolumny_do_zwrotu = ["stacja", "czas_h"]
    if "PM2.5_ug_m3" in polaczone.columns:
        kolumny_do_zwrotu.append("PM2.5_ug_m3")
    if "PM10_ug_m3" in polaczone.columns:
        kolumny_do_zwrotu.append("PM10_ug_m3")
    kolumny_do_zwrotu.extend([col for col in ["temp_srednia", "wilgotnosc_sredni"]

    return polaczone[kolumny_do_zwrotu]

ROOT = Path("dane_lodz/dane_lodz") # Make sure ROOT is correctly defined here to
final = zbuduj_dataset(ROOT)
print("Kształt finalnej ramki:", final.shape)
final.head(10)

```

```
/tmp/ipykernel_11575/3113927470.py:73: FutureWarning: Downcasting object dtype ar
rays on .fillna, .ffill, .bfill is deprecated and will change in a future versio
n. Call result.infer_objects(copy=False) instead. To opt-in to the future behavio
r, set `pd.set_option('future.no_silent_downcasting', True)`
df['wersja'] = df['wersja'].fillna(1).astype(int)
/tmp/ipykernel_11575/3113927470.py:73: FutureWarning: Downcasting object dtype ar
rays on .fillna, .ffill, .bfill is deprecated and will change in a future versio
n. Call result.infer_objects(copy=False) instead. To opt-in to the future behavio
r, set `pd.set_option('future.no_silent_downcasting', True)`
df['wersja'] = df['wersja'].fillna(1).astype(int)
/tmp/ipykernel_11575/3113927470.py:73: FutureWarning: Downcasting object dtype ar
rays on .fillna, .ffill, .bfill is deprecated and will change in a future versio
n. Call result.infer_objects(copy=False) instead. To opt-in to the future behavio
r, set `pd.set_option('future.no_silent_downcasting', True)`
df['wersja'] = df['wersja'].fillna(1).astype(int)
/tmp/ipykernel_11575/3113927470.py:73: FutureWarning: Downcasting object dtype ar
rays on .fillna, .ffill, .bfill is deprecated and will change in a future versio
n. Call result.infer_objects(copy=False) instead. To opt-in to the future behavio
r, set `pd.set_option('future.no_silent_downcasting', True)`
df['wersja'] = df['wersja'].fillna(1).astype(int)
/tmp/ipykernel_11575/3113927470.py:73: FutureWarning: Downcasting object dtype ar
rays on .fillna, .ffill, .bfill is deprecated and will change in a future versio
n. Call result.infer_objects(copy=False) instead. To opt-in to the future behavio
r, set `pd.set_option('future.no_silent_downcasting', True)`
df['wersja'] = df['wersja'].fillna(1).astype(int)
/tmp/ipykernel_11575/3113927470.py:73: FutureWarning: Downcasting object dtype ar
rays on .fillna, .ffill, .bfill is deprecated and will change in a future versio
n. Call result.infer_objects(copy=False) instead. To opt-in to the future behavio
r, set `pd.set_option('future.no_silent_downcasting', True)`
df['wersja'] = df['wersja'].fillna(1).astype(int)
/tmp/ipykernel_11575/3113927470.py:73: FutureWarning: Downcasting object dtype ar
rays on .fillna, .ffill, .bfill is deprecated and will change in a future versio
n. Call result.infer_objects(copy=False) instead. To opt-in to the future behavio
r, set `pd.set_option('future.no_silent_downcasting', True)`
df['wersja'] = df['wersja'].fillna(1).astype(int)
/tmp/ipykernel_11575/3113927470.py:73: FutureWarning: Downcasting object dtype ar
rays on .fillna, .ffill, .bfill is deprecated and will change in a future versio
n. Call result.infer_objects(copy=False) instead. To opt-in to the future behavio
r, set `pd.set_option('future.no_silent_downcasting', True)`
df['wersja'] = df['wersja'].fillna(1).astype(int)
/tmp/ipykernel_11575/3113927470.py:73: FutureWarning: Downcasting object dtype ar
rays on .fillna, .ffill, .bfill is deprecated and will change in a future versio
n. Call result.infer_objects(copy=False) instead. To opt-in to the future behavio
r, set `pd.set_option('future.no_silent_downcasting', True)`
df['wersja'] = df['wersja'].fillna(1).astype(int)
/tmp/ipykernel_11575/3113927470.py:73: FutureWarning: Downcasting object dtype ar
rays on .fillna, .ffill, .bfill is deprecated and will change in a future versio
n. Call result.infer_objects(copy=False) instead. To opt-in to the future behavio
r, set `pd.set_option('future.no_silent_downcasting', True)`
df['wersja'] = df['wersja'].fillna(1).astype(int)
/tmp/ipykernel_11575/3113927470.py:73: FutureWarning: Downcasting object dtype ar
rays on .fillna, .ffill, .bfill is deprecated and will change in a future versio
n. Call result.infer_objects(copy=False) instead. To opt-in to the future behavio
r, set `pd.set_option('future.no_silent_downcasting', True)`
df['wersja'] = df['wersja'].fillna(1).astype(int)
```



```
/tmp/ipykernel_11575/3113927470.py:73: FutureWarning: Downcasting object dtype ar
rays on .fillna, .ffill, .bfill is deprecated and will change in a future versio
n. Call result.infer_objects(copy=False) instead. To opt-in to the future behavio
r, set `pd.set_option('future.no_silent_downcasting', True)`
    df['wersja'] = df['wersja'].fillna(1).astype(int)
/tmp/ipykernel_11575/3113927470.py:73: FutureWarning: Downcasting object dtype ar
rays on .fillna, .ffill, .bfill is deprecated and will change in a future versio
n. Call result.infer_objects(copy=False) instead. To opt-in to the future behavio
r, set `pd.set_option('future.no_silent_downcasting', True)`
    df['wersja'] = df['wersja'].fillna(1).astype(int)
/tmp/ipykernel_11575/3113927470.py:73: FutureWarning: Downcasting object dtype ar
rays on .fillna, .ffill, .bfill is deprecated and will change in a future versio
n. Call result.infer_objects(copy=False) instead. To opt-in to the future behavio
r, set `pd.set_option('future.no_silent_downcasting', True)`
    df['wersja'] = df['wersja'].fillna(1).astype(int)
/tmp/ipykernel_11575/3113927470.py:73: FutureWarning: Downcasting object dtype ar
rays on .fillna, .ffill, .bfill is deprecated and will change in a future versio
n. Call result.infer_objects(copy=False) instead. To opt-in to the future behavio
r, set `pd.set_option('future.no_silent_downcasting', True)`
    df['wersja'] = df['wersja'].fillna(1).astype(int)
/tmp/ipykernel_11575/3113927470.py:73: FutureWarning: Downcasting object dtype ar
rays on .fillna, .ffill, .bfill is deprecated and will change in a future versio
n. Call result.infer_objects(copy=False) instead. To opt-in to the future behavio
r, set `pd.set_option('future.no_silent_downcasting', True)`
    df['wersja'] = df['wersja'].fillna(1).astype(int)
/tmp/ipykernel_11575/3113927470.py:73: FutureWarning: Downcasting object dtype ar
rays on .fillna, .ffill, .bfill is deprecated and will change in a future versio
n. Call result.infer_objects(copy=False) instead. To opt-in to the future behavio
r, set `pd.set_option('future.no_silent_downcasting', True)`
    df['wersja'] = df['wersja'].fillna(1).astype(int)
/tmp/ipykernel_11575/3113927470.py:73: FutureWarning: Downcasting object dtype ar
rays on .fillna, .ffill, .bfill is deprecated and will change in a future versio
n. Call result.infer_objects(copy=False) instead. To opt-in to the future behavio
r, set `pd.set_option('future.no_silent_downcasting', True)`
    df['wersja'] = df['wersja'].fillna(1).astype(int)
/tmp/ipykernel_11575/3113927470.py:73: FutureWarning: Downcasting object dtype ar
rays on .fillna, .ffill, .bfill is deprecated and will change in a future versio
n. Call result.infer_objects(copy=False) instead. To opt-in to the future behavio
r, set `pd.set_option('future.no_silent_downcasting', True)`
    df['wersja'] = df['wersja'].fillna(1).astype(int)
/tmp/ipykernel_11575/3113927470.py:73: FutureWarning: Downcasting object dtype ar
rays on .fillna, .ffill, .bfill is deprecated and will change in a future versio
n. Call result.infer_objects(copy=False) instead. To opt-in to the future behavio
r, set `pd.set_option('future.no_silent_downcasting', True)`
    df['wersja'] = df['wersja'].fillna(1).astype(int)
/tmp/ipykernel_11575/3113927470.py:73: FutureWarning: Downcasting object dtype ar
rays on .fillna, .ffill, .bfill is deprecated and will change in a future versio
n. Call result.infer_objects(copy=False) instead. To opt-in to the future behavio
r, set `pd.set_option('future.no_silent_downcasting', True)`
    df['wersja'] = df['wersja'].fillna(1).astype(int)
/tmp/ipykernel_11575/3113927470.py:73: FutureWarning: Downcasting object dtype ar
rays on .fillna, .ffill, .bfill is deprecated and will change in a future versio
n. Call result.infer_objects(copy=False) instead. To opt-in to the future behavio
r, set `pd.set_option('future.no_silent_downcasting', True)`
    df['wersja'] = df['wersja'].fillna(1).astype(int)
```



```

/tmp/ipykernel_11575/3113927470.py:73: FutureWarning: Downcasting object dtype ar
rays on .fillna, .ffill, .bfill is deprecated and will change in a future versio
n. Call result.infer_objects(copy=False) instead. To opt-in to the future behavio
r, set `pd.set_option('future.no_silent_downcasting', True)`
    df['wersja'] = df['wersja'].fillna(1).astype(int)
/tmp/ipykernel_11575/3113927470.py:73: FutureWarning: Downcasting object dtype ar
rays on .fillna, .ffill, .bfill is deprecated and will change in a future versio
n. Call result.infer_objects(copy=False) instead. To opt-in to the future behavio
r, set `pd.set_option('future.no_silent_downcasting', True)`
    df['wersja'] = df['wersja'].fillna(1).astype(int)
/tmp/ipykernel_11575/3113927470.py:73: FutureWarning: Downcasting object dtype ar
rays on .fillna, .ffill, .bfill is deprecated and will change in a future versio
n. Call result.infer_objects(copy=False) instead. To opt-in to the future behavio
r, set `pd.set_option('future.no_silent_downcasting', True)`
    df['wersja'] = df['wersja'].fillna(1).astype(int)
/tmp/ipykernel_11575/3113927470.py:73: FutureWarning: Downcasting object dtype ar
rays on .fillna, .ffill, .bfill is deprecated and will change in a future versio
n. Call result.infer_objects(copy=False) instead. To opt-in to the future behavio
r, set `pd.set_option('future.no_silent_downcasting', True)`
    df['wersja'] = df['wersja'].fillna(1).astype(int)
/tmp/ipykernel_11575/3113927470.py:73: FutureWarning: Downcasting object dtype ar
rays on .fillna, .ffill, .bfill is deprecated and will change in a future versio
n. Call result.infer_objects(copy=False) instead. To opt-in to the future behavio
r, set `pd.set_option('future.no_silent_downcasting', True)`
    df['wersja'] = df['wersja'].fillna(1).astype(int)
/tmp/ipykernel_11575/3113927470.py:73: FutureWarning: Downcasting object dtype ar
rays on .fillna, .ffill, .bfill is deprecated and will change in a future versio
n. Call result.infer_objects(copy=False) instead. To opt-in to the future behavio
r, set `pd.set_option('future.no_silent_downcasting', True)`
    df['wersja'] = df['wersja'].fillna(1).astype(int)
/tmp/ipykernel_11575/3113927470.py:73: FutureWarning: Downcasting object dtype ar
rays on .fillna, .ffill, .bfill is deprecated and will change in a future versio
n. Call result.infer_objects(copy=False) instead. To opt-in to the future behavio
r, set `pd.set_option('future.no_silent_downcasting', True)`
    df['wersja'] = df['wersja'].fillna(1).astype(int)
/tmp/ipykernel_11575/3113927470.py:73: FutureWarning: Downcasting object dtype ar
rays on .fillna, .ffill, .bfill is deprecated and will change in a future versio
n. Call result.infer_objects(copy=False) instead. To opt-in to the future behavio
r, set `pd.set_option('future.no_silent_downcasting', True)`
    df['wersja'] = df['wersja'].fillna(1).astype(int)
/tmp/ipykernel_11575/3113927470.py:73: FutureWarning: Downcasting object dtype ar
rays on .fillna, .ffill, .bfill is deprecated and will change in a future versio
n. Call result.infer_objects(copy=False) instead. To opt-in to the future behavio
r, set `pd.set_option('future.no_silent_downcasting', True)`
    df['wersja'] = df['wersja'].fillna(1).astype(int)
Kształt finalnej ramki: (160, 6)

```

Out[66]:

	stacja	czas_h	PM2.5_ug_m3	PM10_ug_m3	temp_srednia	wilgotnosc_srednia
0	ST04_gorna	2025-03-01 00:00:00	30.5	36.7	6.650	51.85
1	ST04_gorna	2025-03-01 06:00:00	70.2	116.0	12.905	60.50
2	ST04_gorna	2025-03-01 12:00:00	35.1	46.1	9.875	70.35
3	ST04_gorna	2025-03-01 18:00:00	52.5	64.3	11.355	43.70
4	ST03_baluty	2025-03-05 00:00:00	36.2	46.2	7.315	51.70
5	ST03_baluty	2025-03-05 06:00:00	37.0	64.1	19.185	85.60
6	ST03_baluty	2025-03-05 12:00:00	33.1	43.8	11.935	62.95
7	ST03_baluty	2025-03-05 18:00:00	74.2	93.0	10.875	78.80
8	ST04_gorna	2025-03-08 00:00:00	48.2	80.0	2.140	72.05
9	ST04_gorna	2025-03-08 06:00:00	58.2	89.3	8.470	54.70

